# Processor Prototyping Lab

# Midterm Report

Connor Lai

Michael Wu

Atthin Chandrashekar

October 18, 2026

# 1 Executive Overview

This report explores the implementation of a RISC-V CPU and the performance benefits and technicalities of a pipelined design. The most basic version of a RISC-V CPU completes all five stages—instruction fetch, instruction decode, execute, memory, and writeback—in a single cycle to achieve a net CPI of 1. However, due to its long critical path, it must use lower clock speeds. To improve the performance of the processor, the design is pipelined, splitting up the CPU into its respective stages and executing one stage per cycle. The pipelined design utilizes overlap between instructions to still achieve one instruction per cycle execution in the average case while greatly reducing the clock period. While pipelining improves the overall performance of the CPU, it also introduces many hazards. For one, the CPU cannot both fetch an instruction and access data memory at the same time due to the unified memory architecture. Additionally, the processor needs to handle data hazards from having multiple unfinished instructions in flight simultaneously. Further, jumps and branches require the CPU to erase mis-evaluated instructions and redirect to the correct instructions.

To determine the tangible performance gain from the pipelined design, clock frequency, CPI, total execution time, and FPGA resource utilization were measured and compared between the two processors. By evaluating these metrics across different memory latencies, the effects of pipeline hazards, memory stalls, and increased clock frequency on overall performance could be analyzed. The results showed that the pipelined processor consistently outperformed the single-cycle design, reducing execution time by approximately 16–22% across all tested memory latencies. At zero memory latency, the pipelined processor achieved a maximum operating frequency of 87.63 MHz compared to 50.03 MHz for the single-cycle design, resulting in a 22.2% reduction in execution time. However, this improvement came at the cost of increased hardware complexity and FPGA resource utilization, with 61.2% more slice LUTs and 94.7% more slice registers. Overall, the results demonstrate that pipelining improves processor performance by increasing operating frequency, despite the additional cycles introduced by pipeline hazards and memory stalls.

# 2 Processor Design

## 2.1 Single Cycle

The single-cycle datapath uses one ALU for arithmetic, address calculation, and branch comparisons. Operand selection supports register values and immediates. Multiplexers select operands and the result written to the register file. Memory control unit coordinates access to the shared memory.

## 2.2 Pipelined

In order to convert the single-cycle CPU into a pipelined processor, pipeline registers were added between stages to carry each instruction's data and control information. The pipeline was split into five stages consisting of instruction fetch, decode, execute, memory, and write-back. Forwarding and hazard units support overlapping instructions. The request unit from the single-cycle design was removed from the pipeline as it added an extra stage.

The forwarding unit manages read-after-write dependencies by supplying results from memory or writeback directly to execute. It checks both operands independently and forwards the newest possible result.

The hazard unit manages control hazards, structural hazards, and specifically the load-use data hazard. A load-use dependency holds fetch and decode and inserts an execute-stage bubble. Branch mispredictions and jump redirects flush younger instructions to prevent incorrect updates.

A direct-mapped branch prediction table and branch target buffer stores two-bit saturating counters and branch targets. A valid match predicts the stored target when the counter favors taken. Otherwise fetch continues at $PC + 4$. The original prediction travels with the instruction and is checked in memory against the actual next address. A conflicting branch replaces the entry and initializes its counter from the observed outcome.

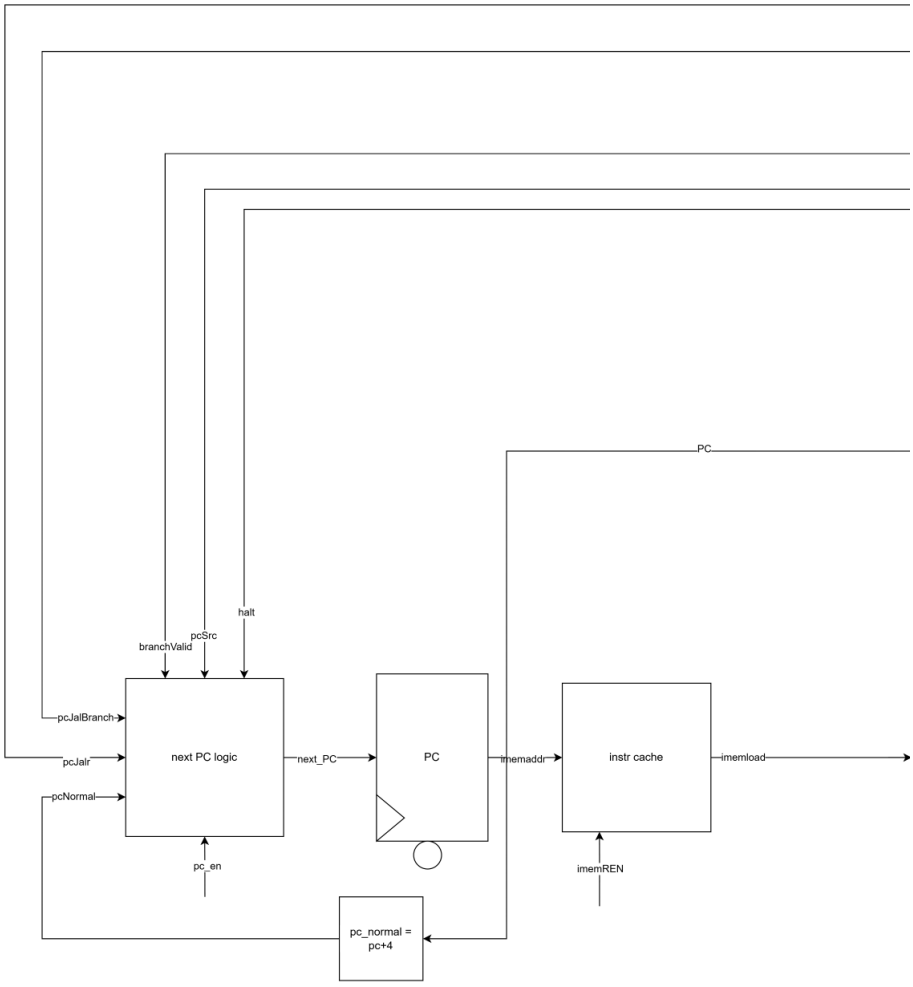



Figure 1: Single-Cycle Processor Datapath: Fetch and Next-PC Selection

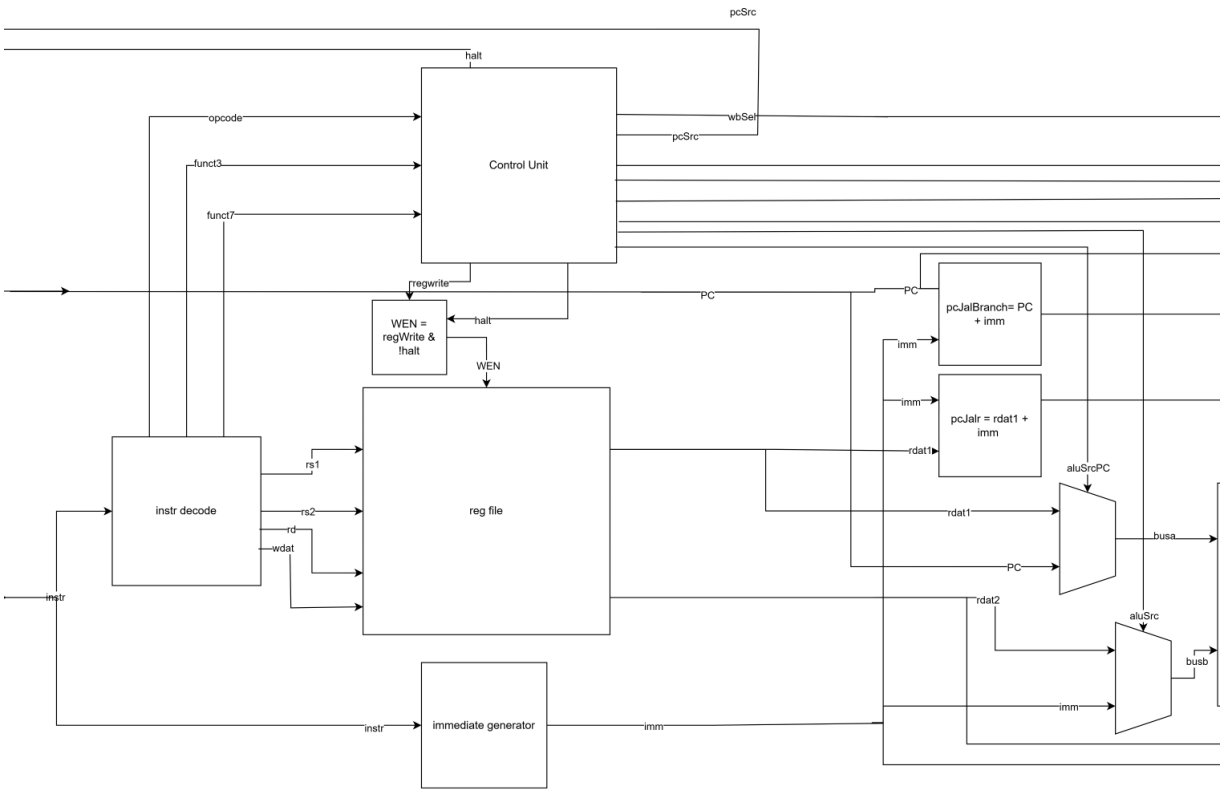



Figure 2: Single-Cycle Processor Datapath: Decode and Operand Selection

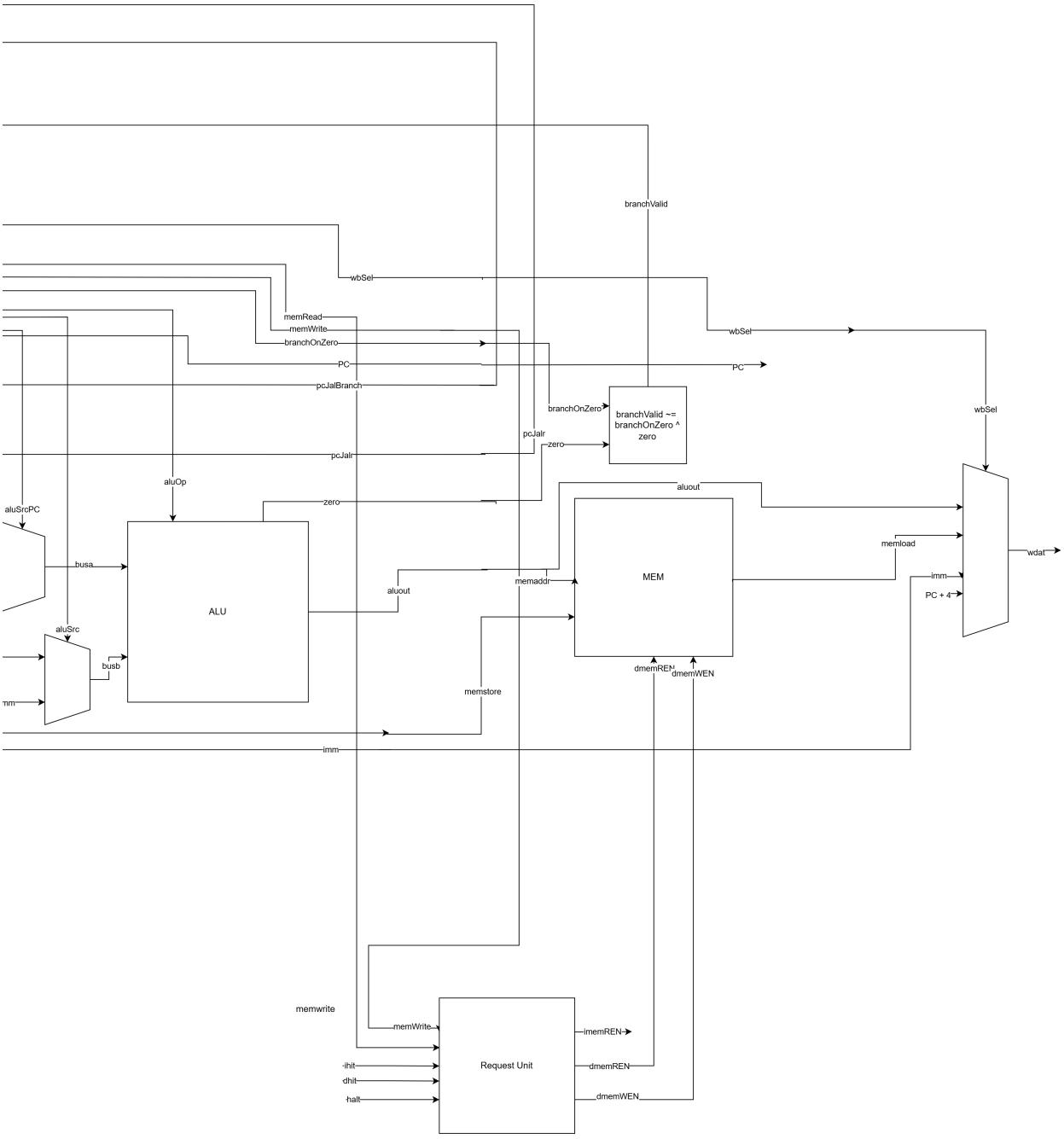



Figure 3: Single-Cycle Processor Datapath: Execute, Memory, and Writeback

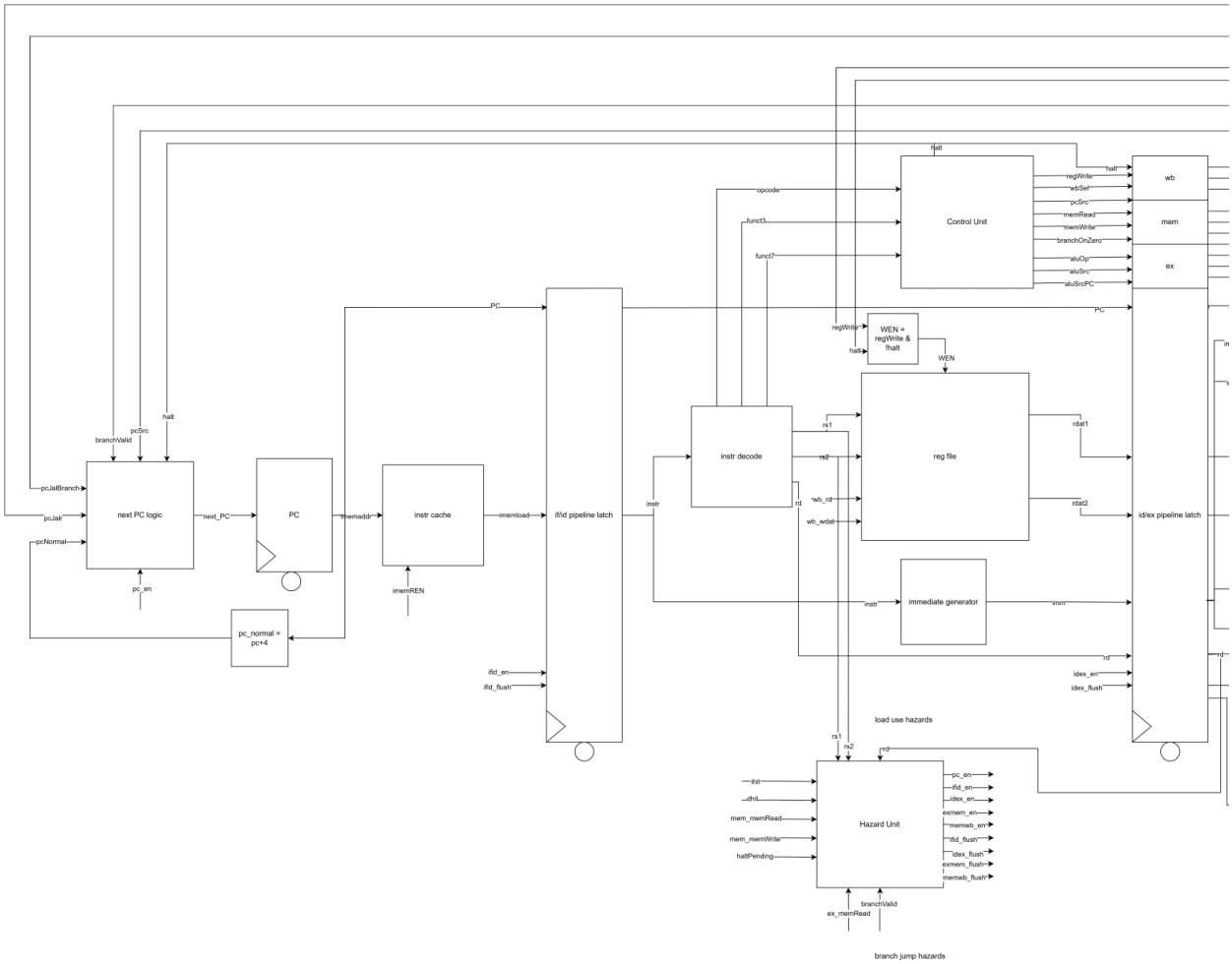



Figure 4: Five-Stage Pipelined Processor Datapath: Fetch and Decode

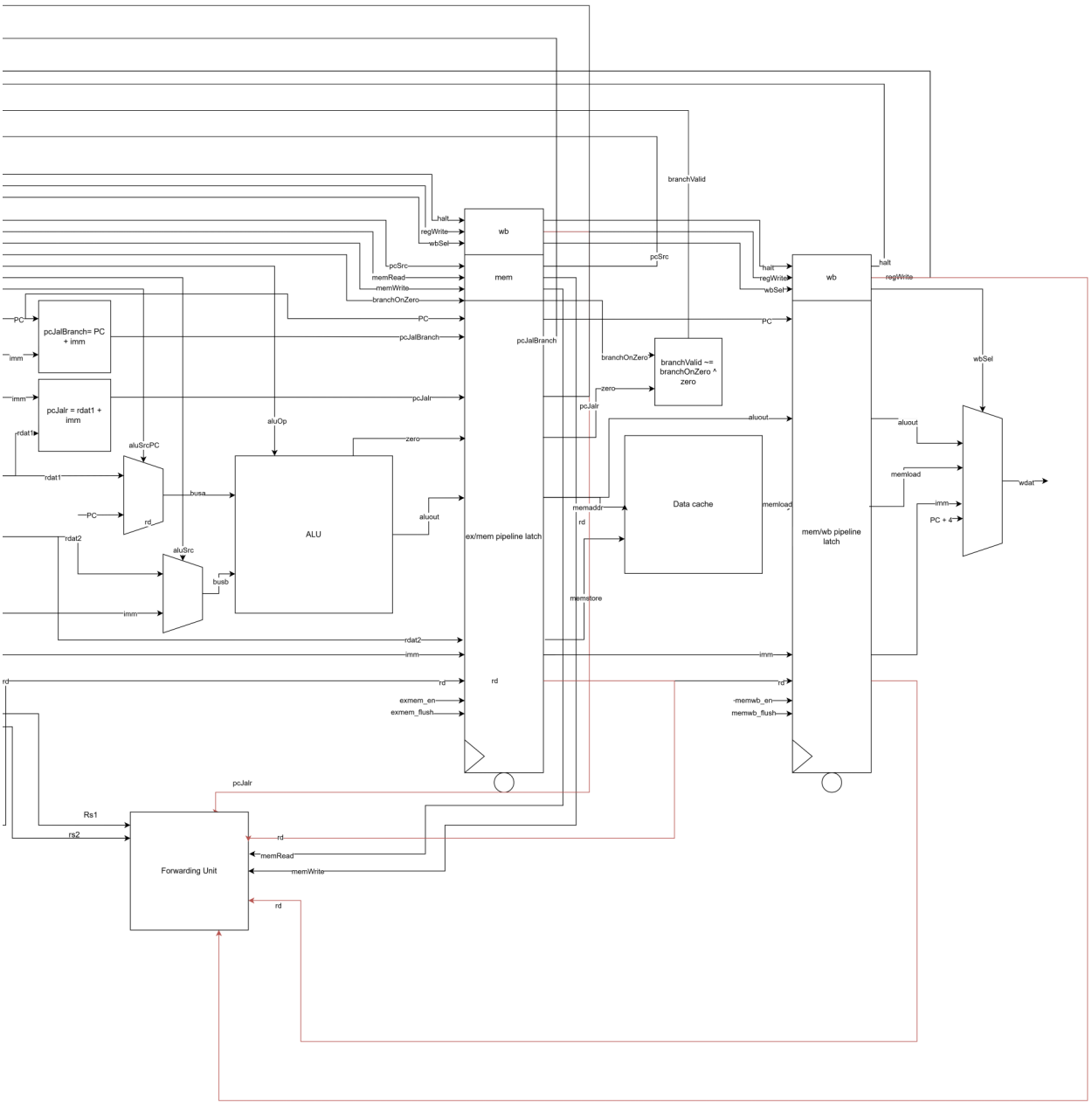



Figure 5: Five-Stage Pipelined Processor Datapath: Execute, MEM, and Writeback

# 3 Results

The single-cycle and pipelined processors were evaluated using the mergesort.asm benchmark at memory latencies of 0, 2, 6, and 10 cycles. Both designs were synthesized and simulated using mapped gate timing. Maximum operating frequencies were obtained from exec_data_sweep.sh, while dynamic instruction counts and CPUCLK cycle counts were obtained from make system.sim. Only CPUCLK cycles were used for CPI and execution time calculations. FPGA resource utilization was obtained from build/utilization.rpt.

The maximum operating frequency was determined by the smaller of half the MAIN frequency and the CPUCLK frequency:

$$f_{\max} = \min\left(\frac{f_{\text{MAIN}}}{2}, f_{\text{CPUCLK}}\right)$$

Average cycles per instruction (CPI) and total execution time were calculated as:

$$\text{CPI} = \frac{N_{\text{cycles}}}{N_{\text{instructions}}} \qquad T_{\text{exec}}(\mu s) = \frac{N_{\text{cycles}}}{f_{\max}(\text{MHz})}$$

The ideal instruction latency was calculated using the clock period and the number of pipeline stages:

$$T_{\text{ideal,single}}(\text{ns}) = \frac{1000}{f_{\max}(\text{MHz})}$$

$$T_{\text{ideal,pipeline}}(\text{ns}) = \frac{5000}{f_{\max}(\text{MHz})}$$

These metrics were used to compare execution performance, memory latency sensitivity, and hardware resource requirements between the two architectures.

## 3.1 Single Cycle

The single-cycle processor was evaluated using the mergesort.asm benchmark at memory latencies of 0, 2, 6, and 10 cycles. Maximum operating frequency was determined from

synthesis timing reports using $f_{\max} = \min(f_{\text{MAIN}}/2, f_{\text{CPUCLK}})$. CPU cycle counts and dynamic instruction counts were obtained from simulation, while FPGA resource utilization was obtained from the synthesis reports. Table 1 summarizes the results.

| **Specification** | **LAT = 0** | **LAT = 2** | **LAT = 6** | **LAT = 10** |
|---|---|---|---|---|
| MAIN Fmax (MHz) | 100.35 | 100.03 | 100.03 | 100.03 |
| CPUCLK Fmax (MHz) | 50.03 | 50.09 | 50.09 | 50.09 |
| Effective Fmax (MHz) | 50.03 | 50.015 | 50.015 | 50.015 |
| CPU Cycles | 6907 | 13814 | 27628 | 41442 |
| Instructions | 5409 | 5409 | 5409 | 5409 |
| Average CPI | 1.277 | 2.554 | 5.108 | 7.662 |
| Ideal Latency (ns) | 19.988 | 19.994 | 19.994 | 19.994 |
| Execution Time ($\mu$s) | 138.06 | 276.20 | 552.39 | 828.59 |
| Slice LUTs | 1866 | 1866 | 1866 | 1866 |
| Slice Registers | 1180 | 1180 | 1180 | 1180 |
| F7 Muxes | 410 | 410 | 410 | 410 |
| F8 Muxes | 110 | 110 | 110 | 110 |

Table 1: Single-Cycle Processor Performance and FPGA Utilization

The single-cycle processor maintained an operating frequency of approximately 50 MHz across all memory latency configurations. However, CPI increased from 1.277 at LAT = 0 to 7.662 at LAT = 10, resulting in a sixfold increase in execution time from 138.06 to 828.59 $\mu$s. The instruction count of 5,409 indicates that the additional execution cycles were caused by memory-related delays rather than changes in program execution. These results demonstrate that memory latency significantly affects processor performance despite a relatively constant operating frequency.

## 3.2 Pipelined

The five-stage pipelined processor was evaluated using the same mergesort.asm benchmark at memory latencies of 0, 2, 6, and 10 cycles. Table 2 summarizes the processor's performance

and FPGA resource utilization.

| **Specification** | **LAT = 0** | **LAT = 2** | **LAT = 6** | **LAT = 10** |
|---|---|---|---|---|
| MAIN Fmax (MHz) | 177.43 | 143.39 | 143.39 | 143.39 |
| CPUCLK Fmax (MHz) | 87.63 | 71.695 | 71.695 | 71.695 |
| Effective Fmax (MHz) | 87.63 | 71.695 | 71.695 | 71.695 |
| CPU Cycles | 9408 | 16451 | 33197 | 47011 |
| Instructions | 5409 | 5409 | 5409 | 5409 |
| Average CPI | 1.739 | 3.041 | 6.137 | 8.691 |
| Ideal Latency (ns) | 57.06 | 69.74 | 69.74 | 69.74 |
| Execution Time ($\mu$s) | 107.36 | 229.46 | 463.03 | 655.71 |
| Slice LUTs | 3008 | 3008 | 3008 | 3008 |
| Slice Registers | 2297 | 2297 | 2297 | 2297 |
| F7 Muxes | 453 | 453 | 453 | 453 |
| F8 Muxes | 64 | 64 | 64 | 64 |

Table 2: Pipelined Processor Performance and FPGA Utilization

The pipelined processor achieved a maximum operating frequency of 87.63 MHz at LAT = 0, compared to approximately 50 MHz for the single-cycle processor. Despite its higher CPI of 1.739, the shorter clock period reduced total execution time to approximately 107.36 $\mu$s. As memory latency increased, CPI rose to 8.691 at LAT = 10, increasing execution time to 655.71 $\mu$s. The higher operating frequency allowed the pipelined processor to maintain lower execution times than the single-cycle design across all tested memory latencies.

## 3.3 Performance Comparison

The pipelined processor outperformed the single-cycle processor across all memory latencies despite having a higher CPI. Its higher operating frequency reduced execution time by 22.2% at LAT = 0 and 20.9% at LAT = 10. Table 3 summarizes the performance improvements.

| LAT | Single ($\mu$s) | Pipeline ($\mu$s) | Speedup | Reduction |
|---|---|---|---|---|
| 0 | 138.06 | 107.36 | 1.286 | 22.2% |
| 2 | 276.20 | 229.46 | 1.204 | 16.9% |
| 6 | 552.39 | 463.03 | 1.193 | 16.2% |
| 10 | 828.59 | 655.71 | 1.264 | 20.9% |

Table 3: Single-Cycle and Pipelined Processor Performance Comparison

Although pipeline stalls and hazards increased CPI, the higher clock frequency compensated for this, resulting in execution time reductions of approximately 16–22% across the tested memory latencies.

| Resource | Single-Cycle | Pipelined | Change |
|---|---|---|---|
| Slice LUTs | 1866 | 3008 | +61.2% |
| Slice Registers | 1180 | 2297 | +94.7% |
| F7 Muxes | 410 | 453 | +10.5% |
| F8 Muxes | 110 | 64 | -41.8% |

Table 4: FPGA Resource Utilization Comparison

The performance improvement comes at the cost of additional FPGA resources, with the pipelined processor requiring 61.2% more slice LUTs and 94.7% more slice registers. This increased resource utilization reflects the additional pipeline registers, forwarding logic, hazard control, and branch prediction logic required for pipelined execution.

# 4 Conclusion

The mergesort benchmark showed that the pipelined processor completed execution faster than the single-cycle processor at every tested memory latency. Although the CPI of the pipelined processor was higher due to hazards and stalls, the increased clock frequency compensated for the extra cycles. Its operating frequency increased from 50.03 to 87.63

MHz, which outweighed the increase in CPI from 1.277 to 1.739. Overall, the execution time was reduced by 16–22%, with the largest reduction occurring at LAT = 0.

The increase in performance came at the cost of resource utilization. The pipeline used 3008 LUTs and 2297 registers, compared to the 1866 LUTs and 1180 registers used for the single-cycle processor. This corresponds to increases of 61.2% and 94.7%. Comparing this to the speedup of 1.19×–1.29× provided by the pipeline, it is clear that the resource increases outpaced the performance gained and that there is a substantial tradeoff between hardware and performance. Overall, under the tested conditions, the pipeline favors execution speed, while the single-cycle processor favors compactness and implementation simplicity.

# 5 Contributions

**Connor Lai:** Contributed to the development and verification of the pipelined processor, including the forwarding unit, hazard unit testbench, overall datapath integration, debugging, and testing. Also assisted with performance measurements for CPI and execution time, synthesis, and analysis of results.

**Michael Wu:** Contributed to the development of the single-cycle CPU design and pipelined processor. This includes the hazard unit, forwarding unit testbench, pipeline control, and overall datapath integration, debugging, and testing. Also assisted with performance analysis and synthesis.